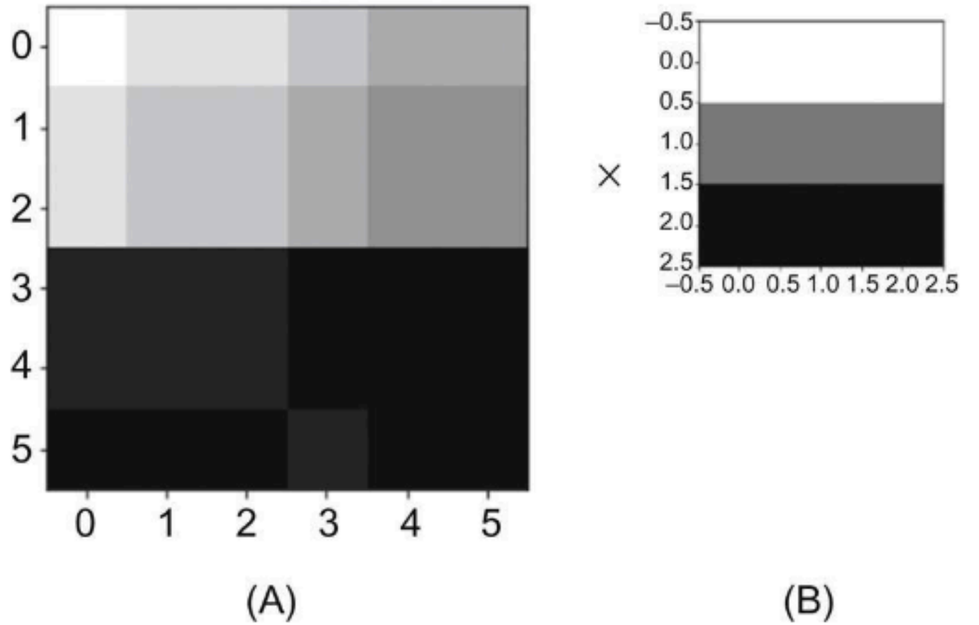
Natural Language Processing
Lab 8: Neural Models - CNN

Lonny Chen: l.chen@hertie-school.org
GRAD-E1282, Fall 2026
28-29 October 2026

Image Convolution (2D)

$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

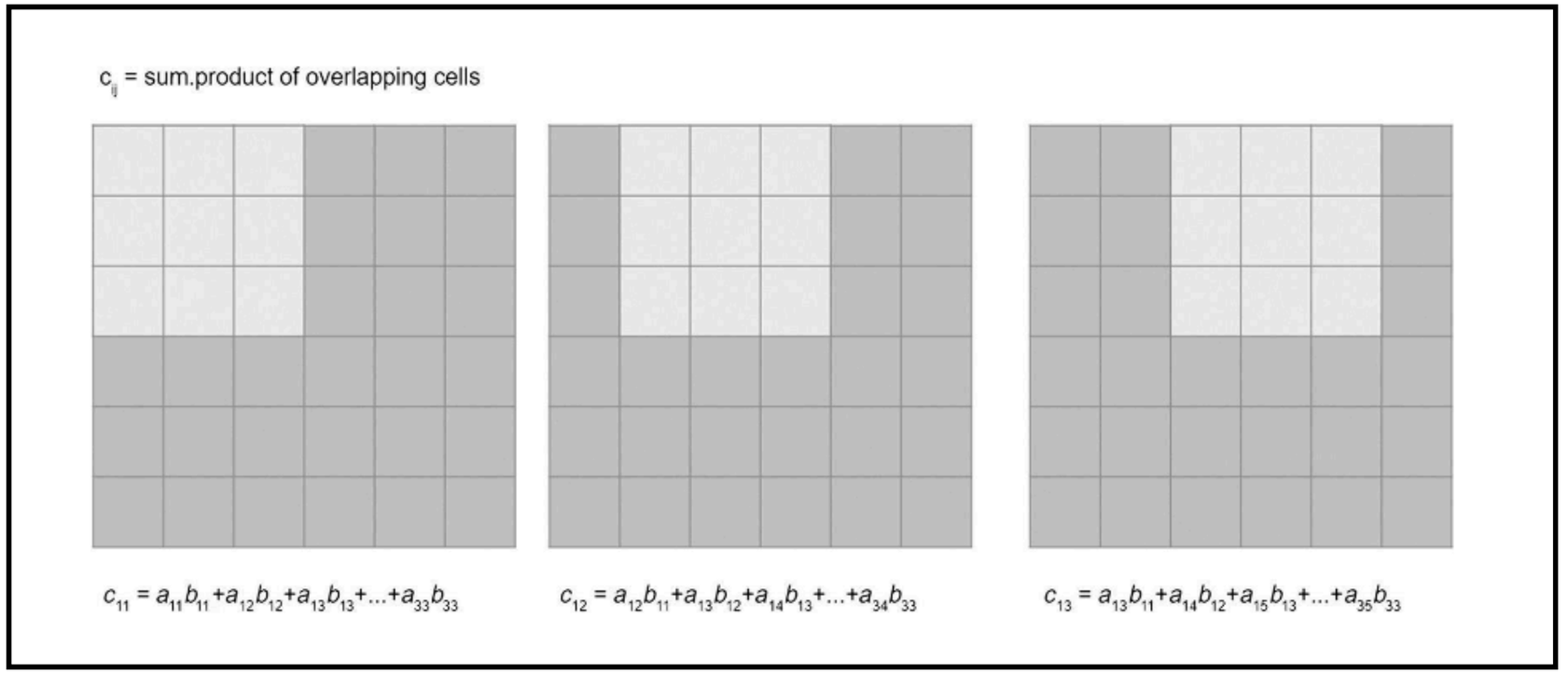


Source: Kotu & Deshpande, 2019

Image Convolution (2D)

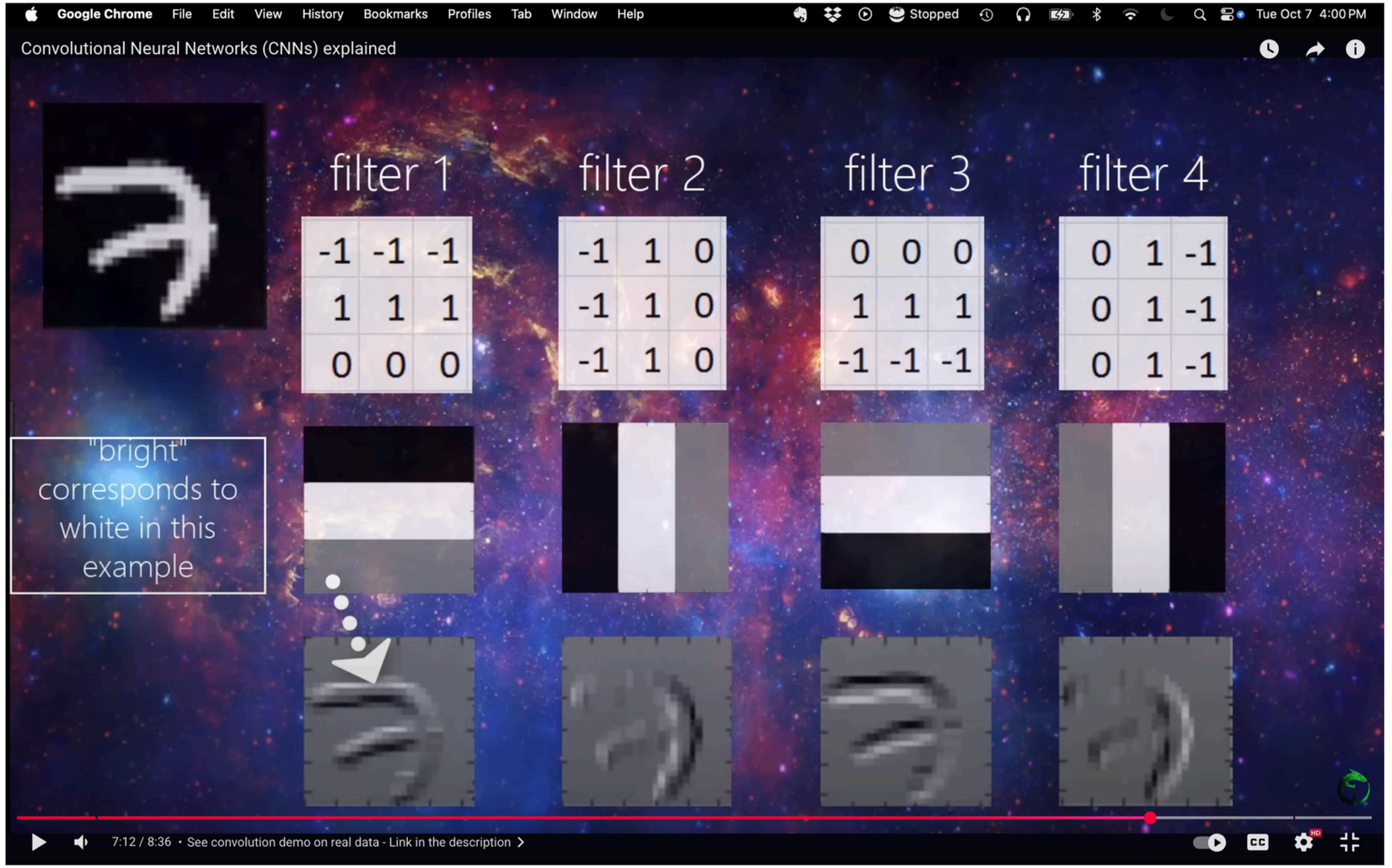
$$A = \begin{bmatrix} 10 & 9 & 9 & 8 & 7 & 7 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 9 & 8 & 8 & 7 & 6 & 6 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



Source: Kotu & Deshpande, 2019

Image Convolution (2D)



Source: deeplizard, 2017

Image Convolution (2D)

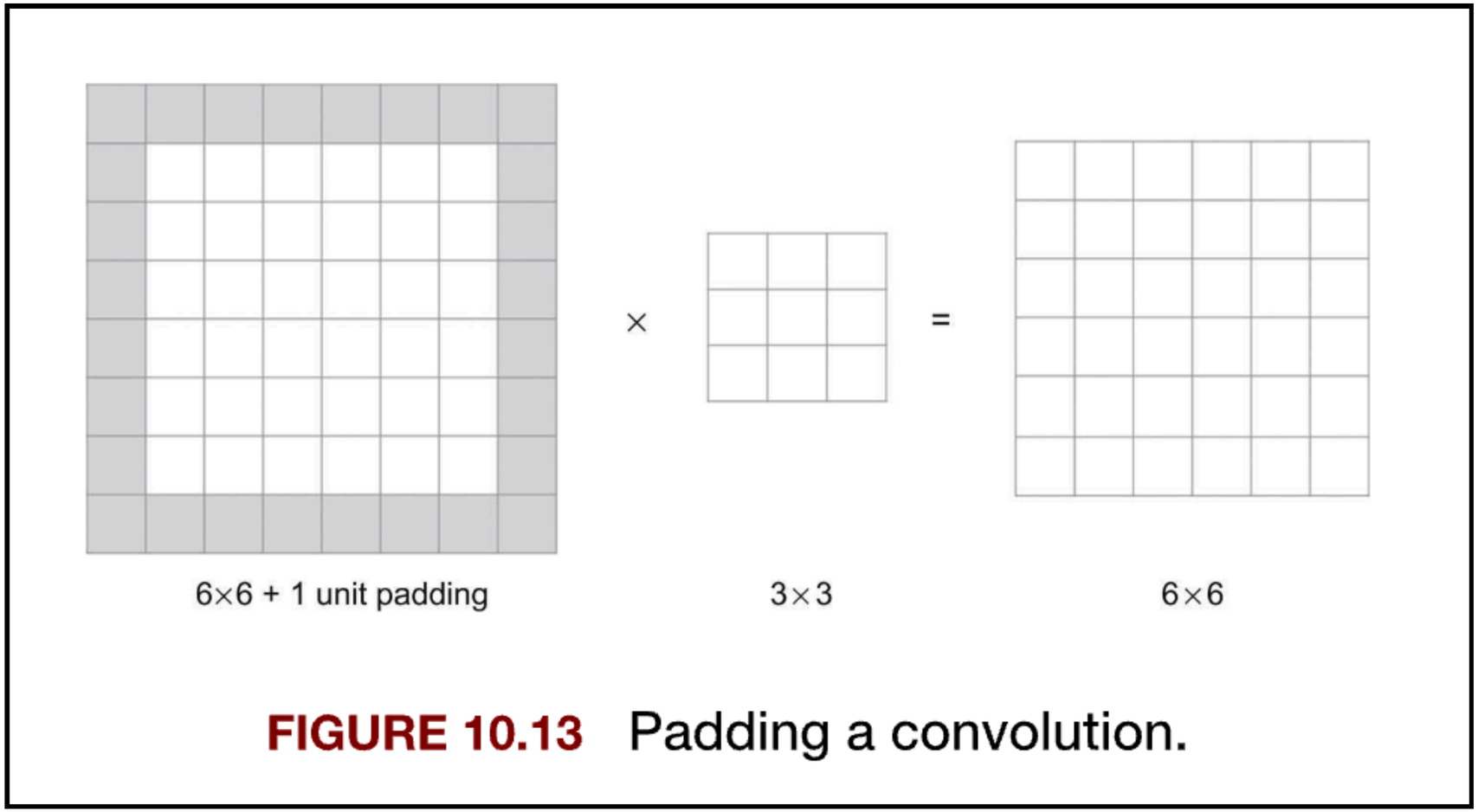


FIGURE 10.13 Padding a convolution.

Source: Kotu & Deshpande, 2019

Image Convolution (2D)

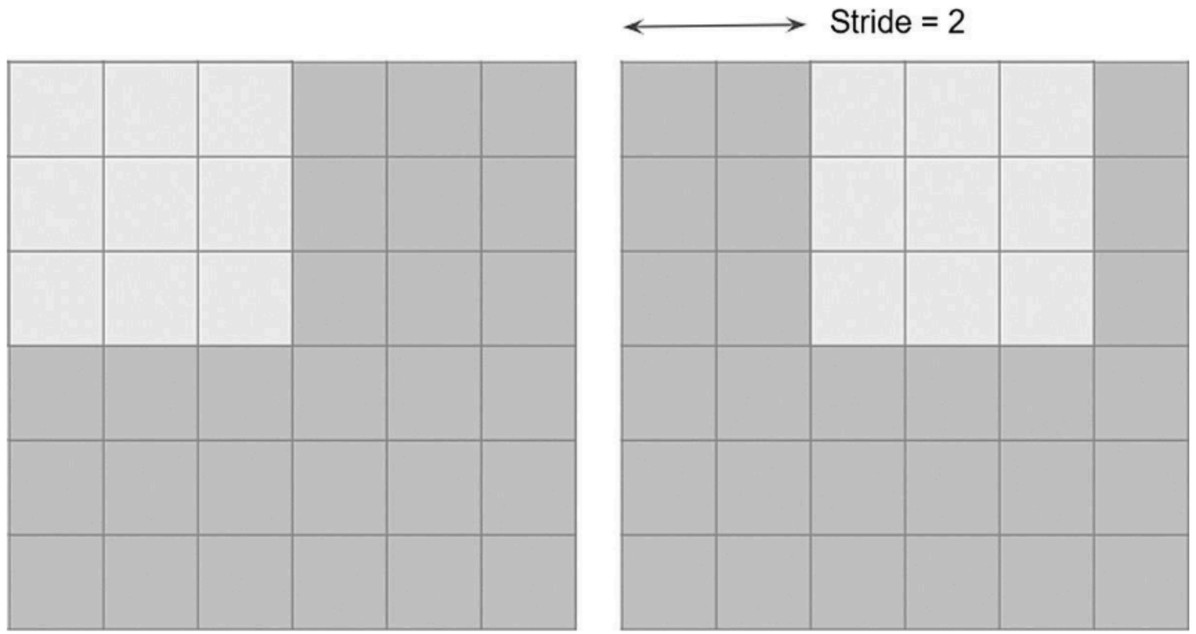


FIGURE 10.14 Visualizing the stride during a convolution.

Source: Kotu & Deshpande, 2019

Image Convolution (2D)

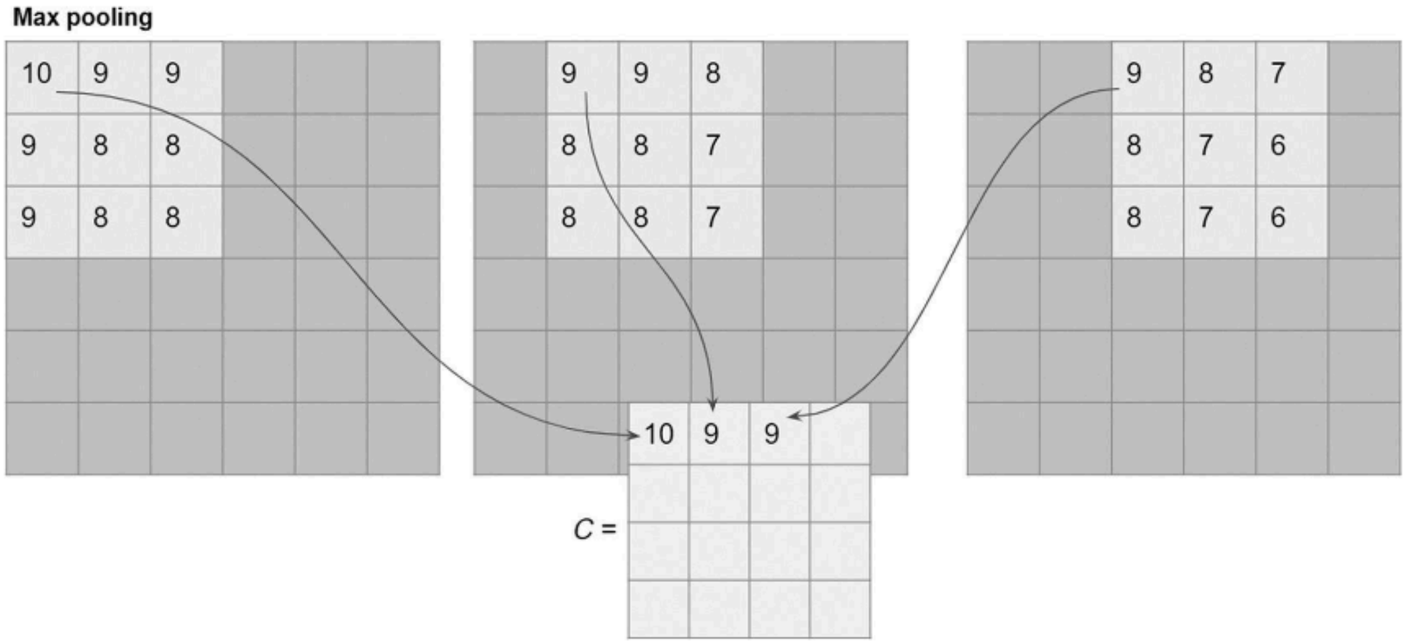


FIGURE 10.15 Max pooling visualized.

Source: Kotu & Deshpande, 2019

Image Convolution (2D)

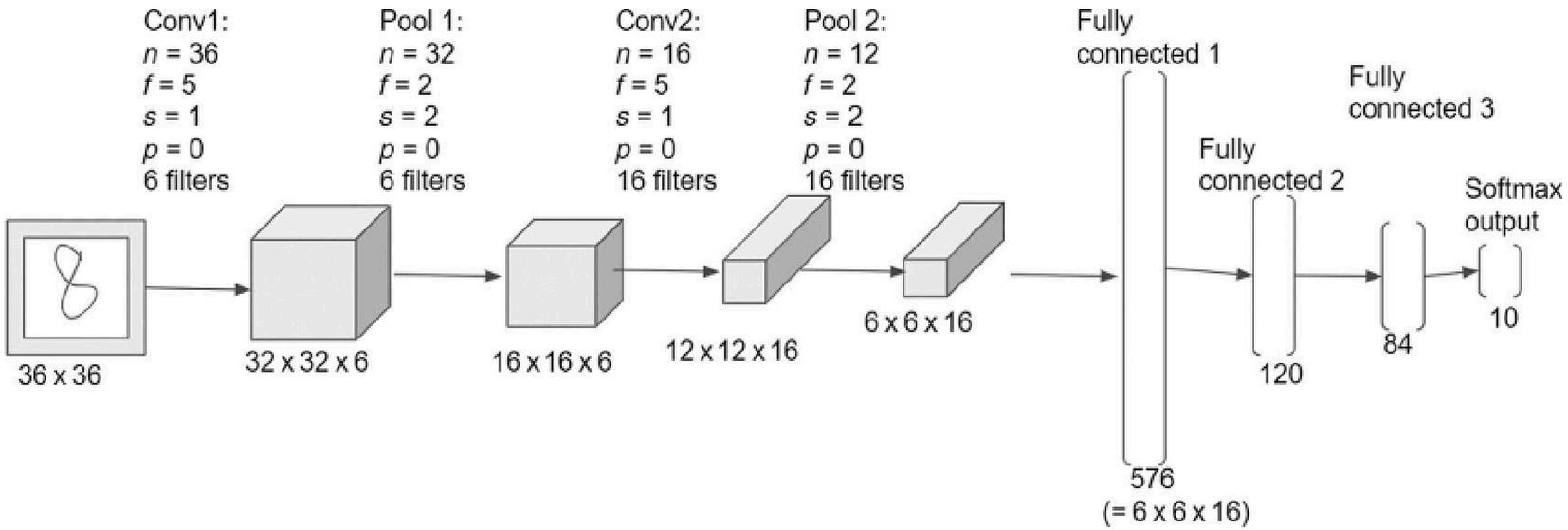
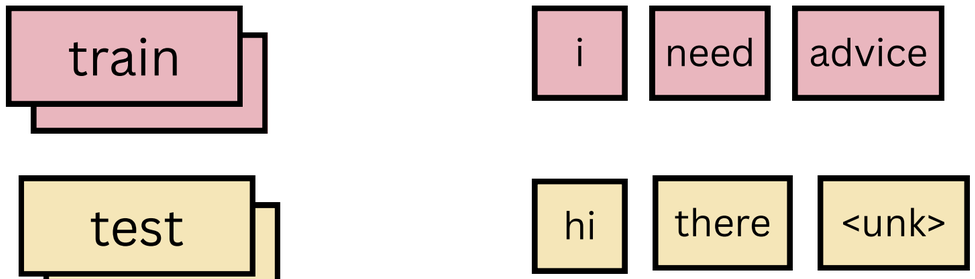


FIGURE 10.18 Architecture of a typical CNN model.

Source: Kotu & Deshpande, 2019

PyTorch: Data and Embeddings

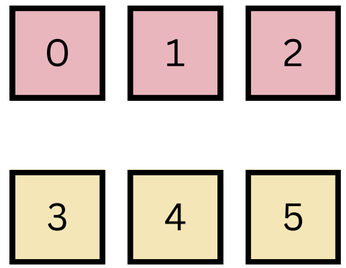
1. Prepare → 2. Tokenize



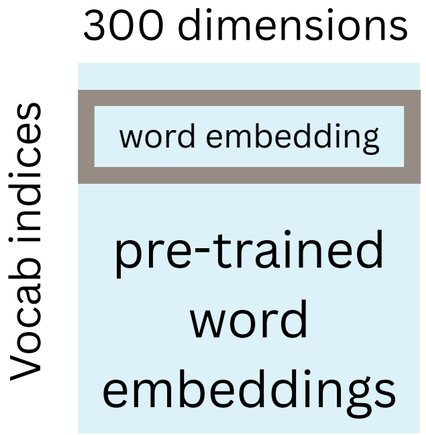
3. Vocabulary

```
{'i': 0,
 'need': 1,
 'advice': 2,
 'hi': 3,
 'there': 4,
 '<unk>': 5, ...
```

4. Encode, truncate, pad (collate_fn)



5. Embedding Matrix



PyTorch: Model Layers

```
classCNNModel(nn.Module):
    def __init__
```

nn.Embedding

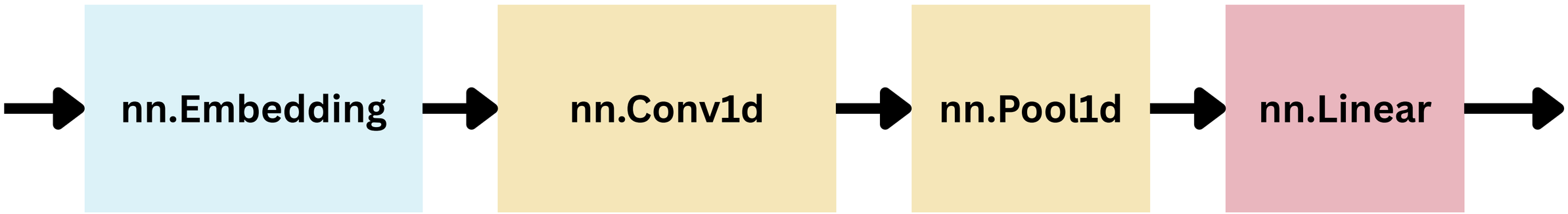
nn.Conv1d

nn.Pool1d

nn.Linear

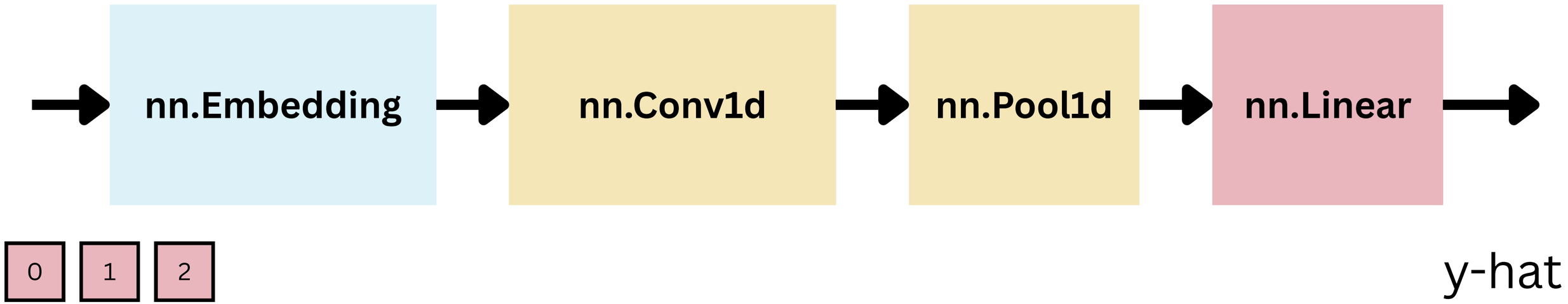
PyTorch: Model Data Flow

```
classCNNModel(nn.Module):
    def forward
```



PyTorch: Model Data Flow

```
classCNNModel(nn.Module):
    def forward
```



Next Week: More Neural Models!

